Memory and Programmable Logic

Outline

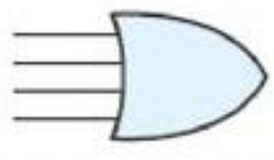
- **RAM**
- Memory decoding
- **ROM**
- Programmable Logic Array (PLA)
- Programmable Array Logic (PAL)

Memories

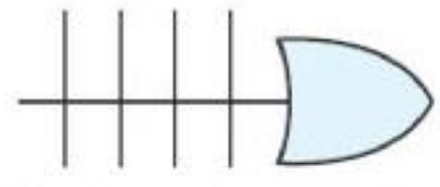
- Memory unit is a collection of cells capable of storing a large quantity of binary information.
  - Information from input device is stored in memory
  - Information to output is taken from memory
- Two types of memories
  - Random-access memory (RAM)
    - Stores binary info for later use
    - *Write* operation: Storing data into memory
    - *Read* operation: Transferring data out of memory
  - Read-only memory (ROM)
    - Can perform only read operation

RAM vs ROM

- ROM is a *programmable logic device* (PLO).
  - The binary information is embedded within the hardware.
- Other programmable devices
  - Programmable logic array (PLA)
  - Programmable array logic (PAL)
  - Field programmable gate array (FPGA)
- Since PLDs have a lot of gates and paths, gates in array logic is shown in different way.



(a) Conventional symbol



(b) Array logic symbol

Random-Access Memory (RAM)

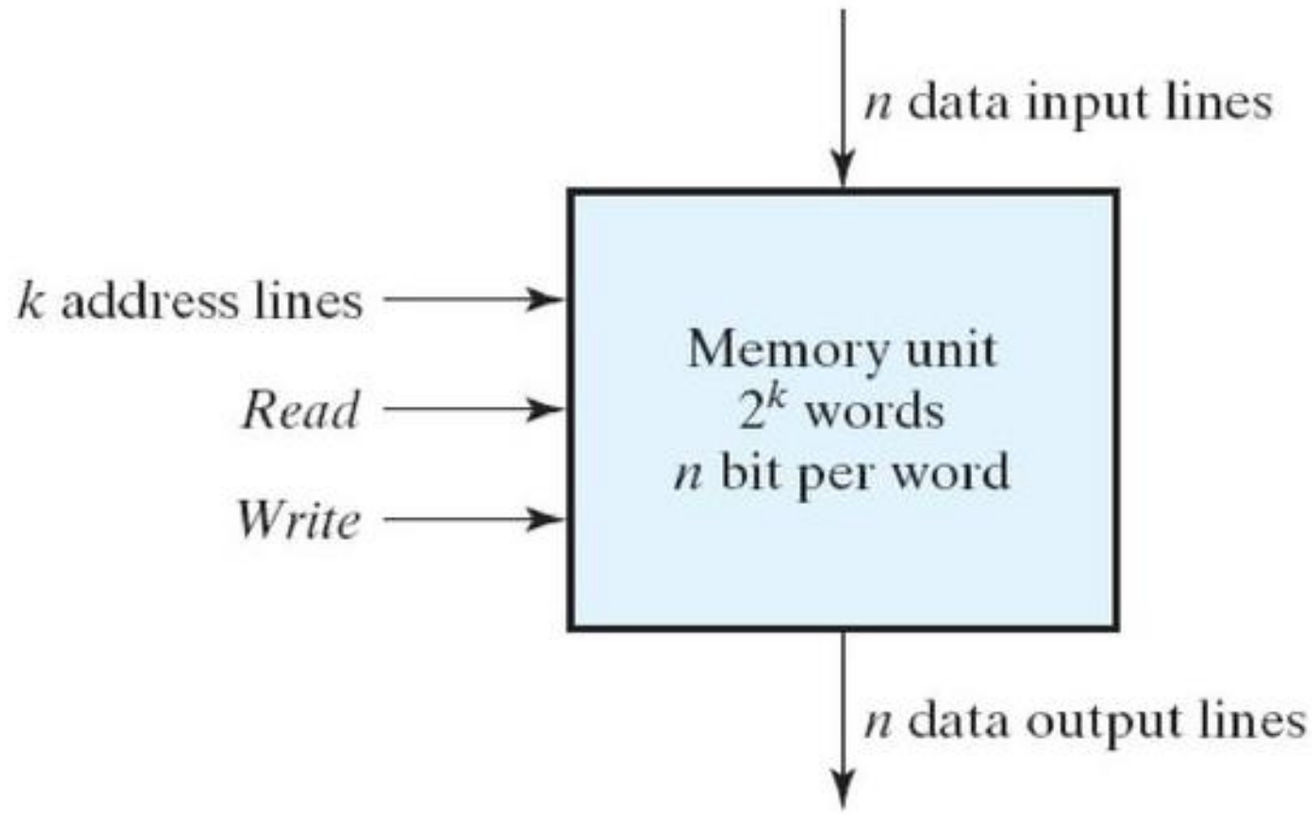
- The time it takes to transfer to or from any random location always the same
  - That's why the name *random access memory.*
  - In magnetic tapes, the time depends on the location of the data.

Word size

- 4 bits - nibble
- 8 bits - byte
- 16 bits - 2 bytes
- 32 bits - 4 bytes
- Most computer uses multiples of 8 bits
- The capacity of a memory is stated as the total number of bytes.

A memory unit

- Communication between memory and its environment is achieved by
  - Data input and output lines
  - Address selection lines
  - Control lines



Address

- Each word is assigned to an address
  - Ranges from 0 to 2^k-1, where k is the number of address lines
- Internal decoder decodes the address for specific word.
- Memory size vs address lines
  - 10 bits (k=10) can address 2^{10} words
  - 32 bits, 2^{32} words

Memory addressing

- A memory with 1K words of 16 bits each

Memory address		Memory content
Binary	Decimal	
0000000000	0	1011010101011101
0000000001	1	1010101110001001
0000000010	2	0000110101000110
	⋮	⋮
1111111101	1021	1001110100010100
1111111110	1022	0000110100011110
1111111111	1023	1101111000100101

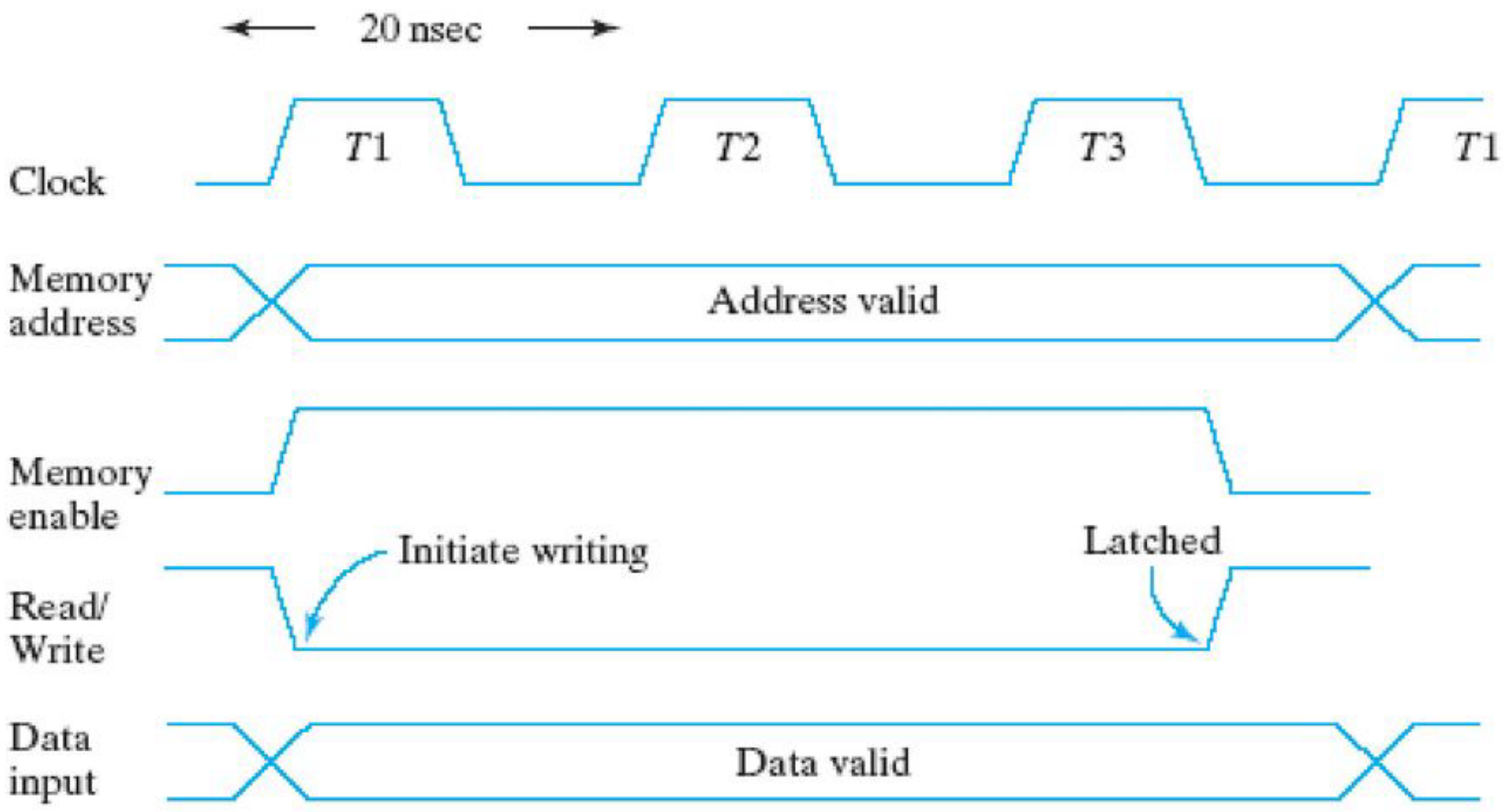
Read and Write Operations

- To read data
  - Put the binary address on the address lines
  - Activate *read* input
- To write data
  - Put binary address on the address lines
  - Put data on data input lines
  - Activate *write* input

Control Inputs to Memory Chip

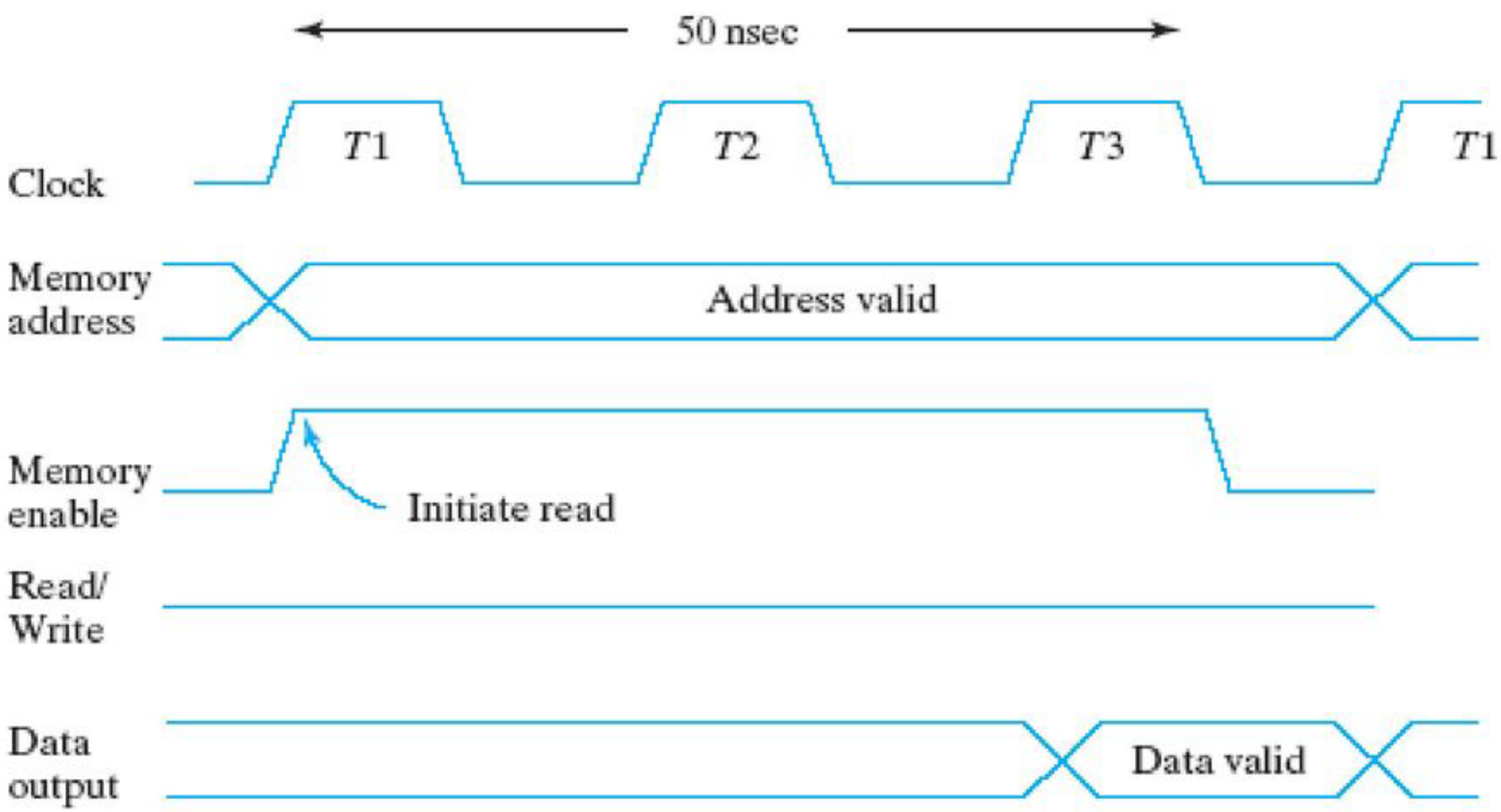
Memory Enable	Read/Write	Memory Operation
0	X	None
1	0	Write to selected word
1	1	Read from selected word

Write cycle



(a) Write cycle

Read cycle



(b) Read cycle

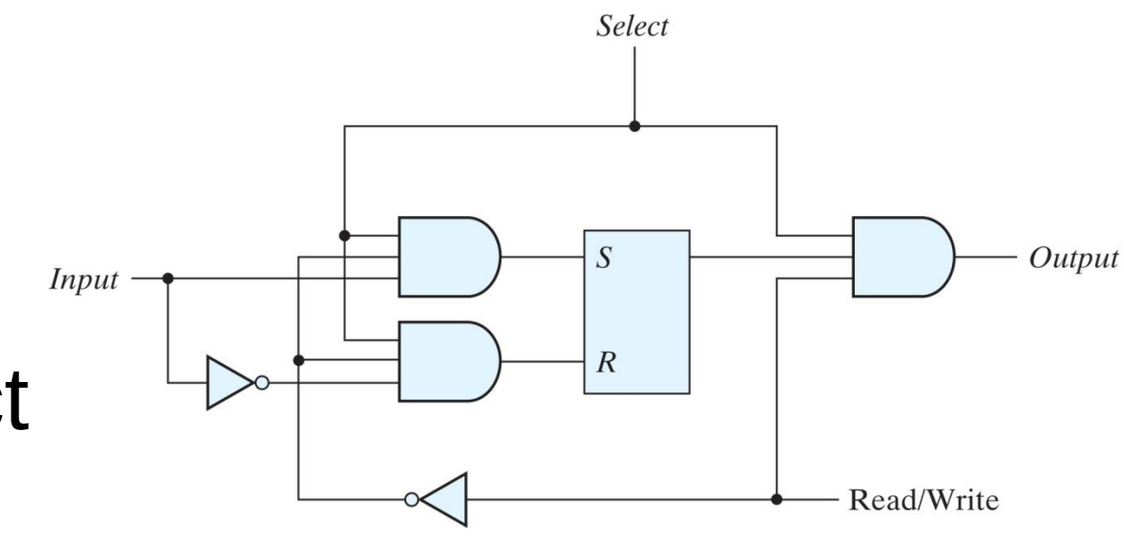
Types of memories

- Static RAM (SRAM)
  - Consists of latches to store binary data
  - Stored data is valid as long as power is applied
  - Easier to use
  - Shorter write and read cycles
- Dynamic RAM (DRAM)
  - Stores data in the form of electric charges on capacitors (MOS transistors).
  - It must be refreshed periodically.
  - Has less power consumption
  - Larger storage capacity.

Volatile vs Nonvolatile

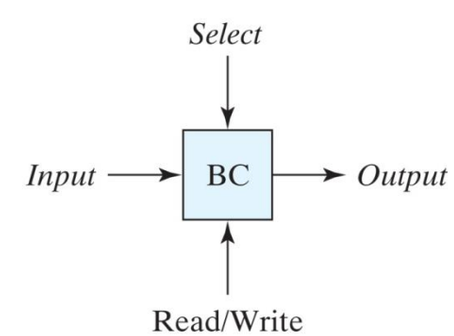
- SRAMs and DRAMs are volatile memories
  - Since they loose data when power is turned off
- Magnetic disks are nonvolatile
  - They store data using magnetization.

Memory Decoding



(a) Logic diagram

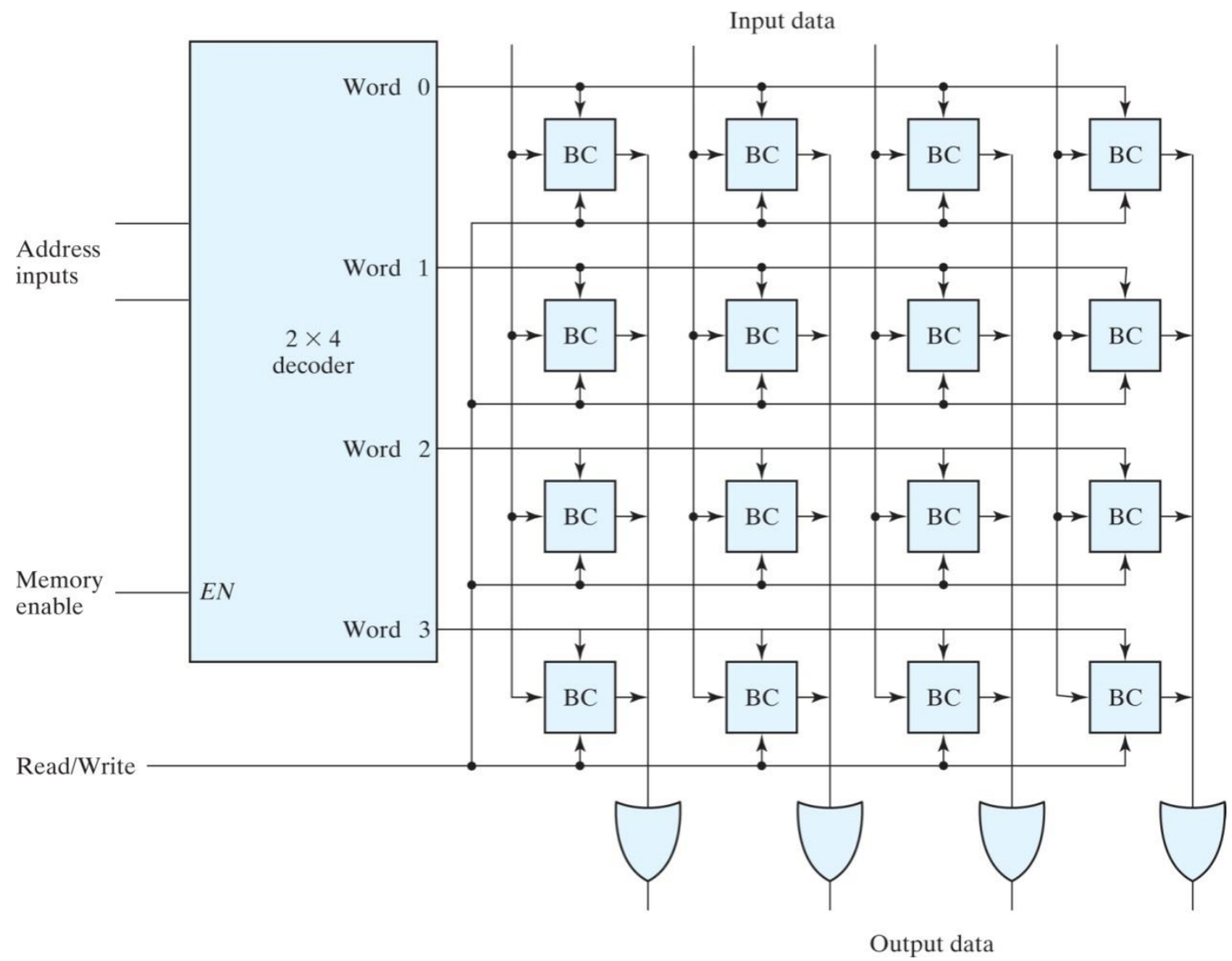
- Decoders are used to select word locations.
- A memory with m words and n bits per word requires mxn storage cells.
- A basic cell is behaves like D latch (4 to 6 transistors)
  - When read/write=1, read operation
  - When read/write=O, write operation



(b) Block diagram

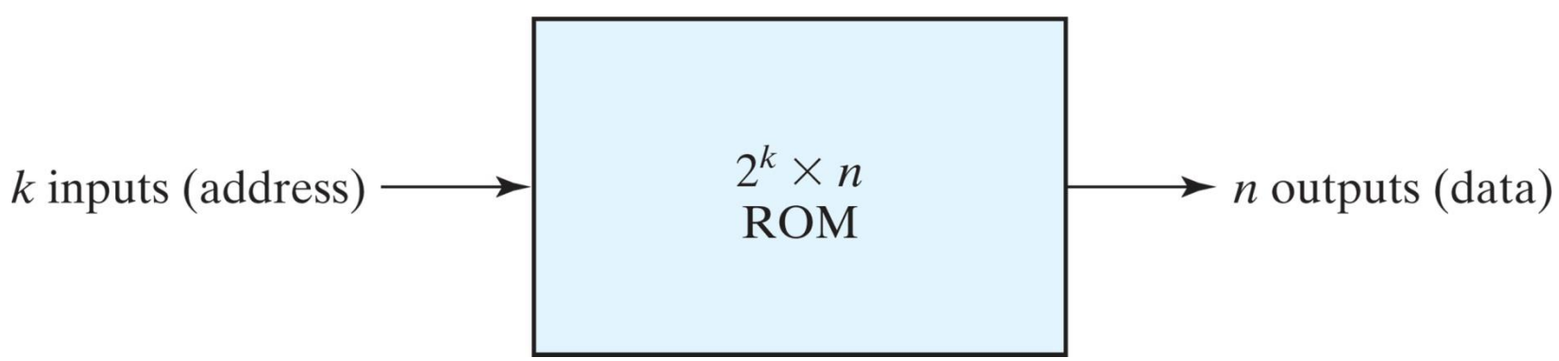
4x4RAM

- 4 words needs 2 address lines to be decoded
- 2^k words needs k address lines



ROM

- Only read occurs
- K inputs and n outputs
- Nonvolatile



32x8 bit ROM

- 32 words of 8 bits each.
- 2^kxn ROM has kx2^k decoder and n OR gates

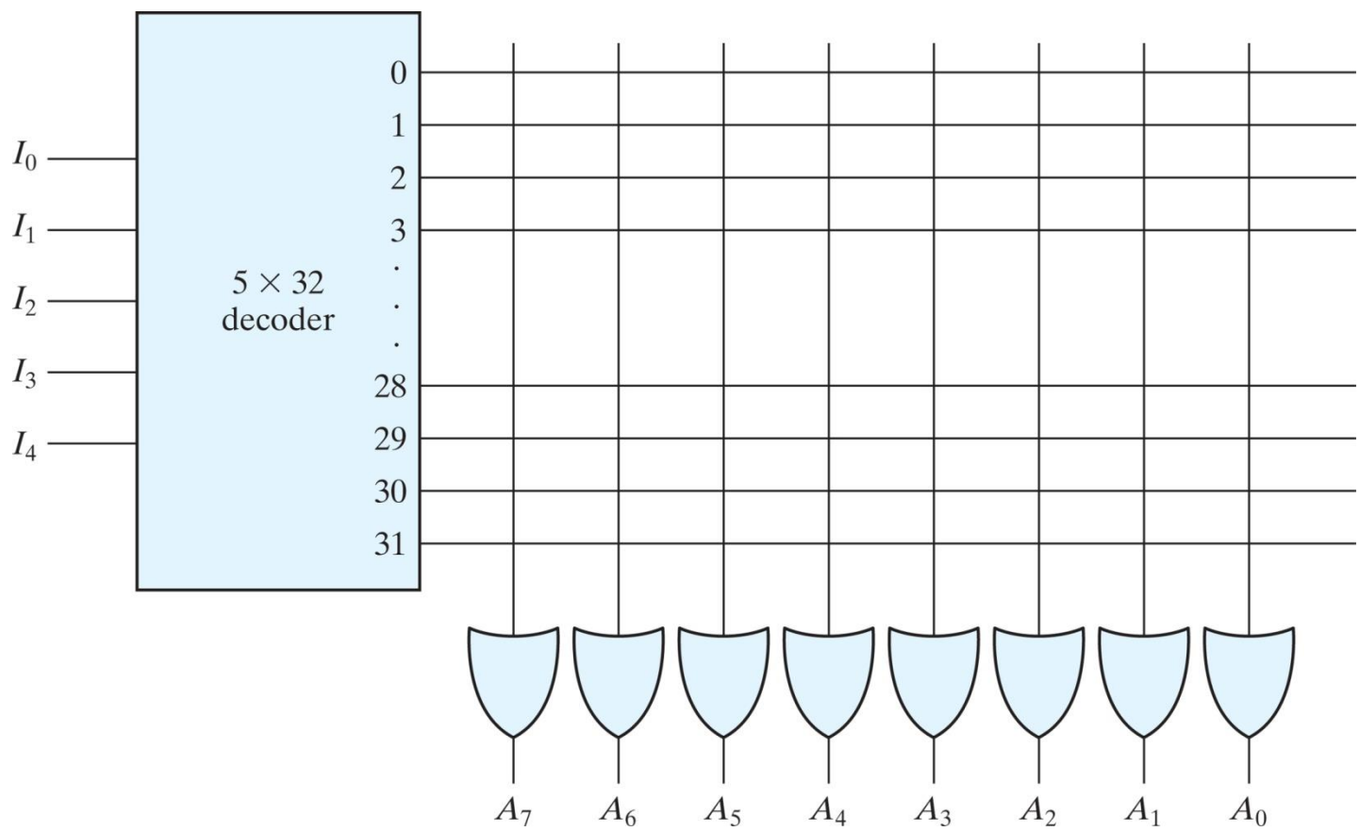
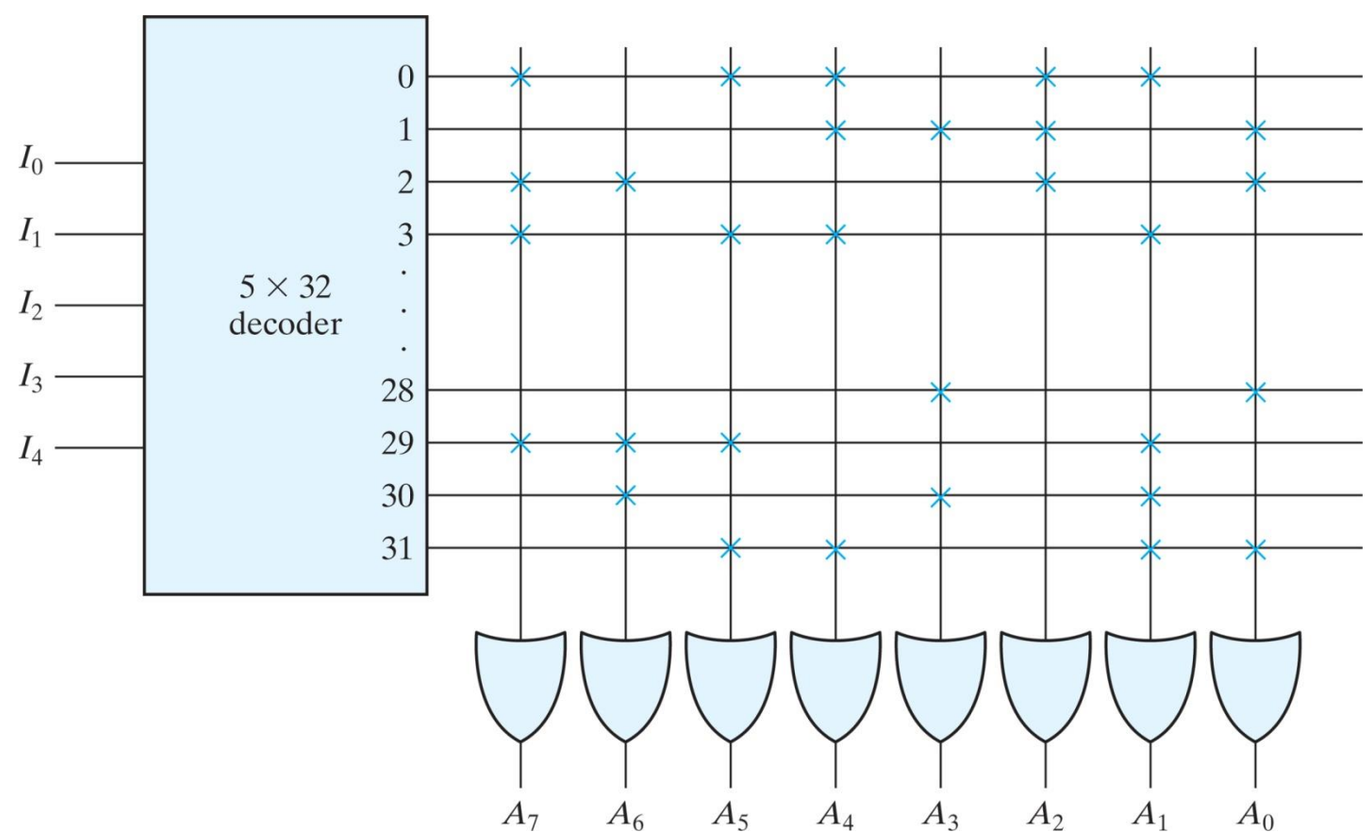


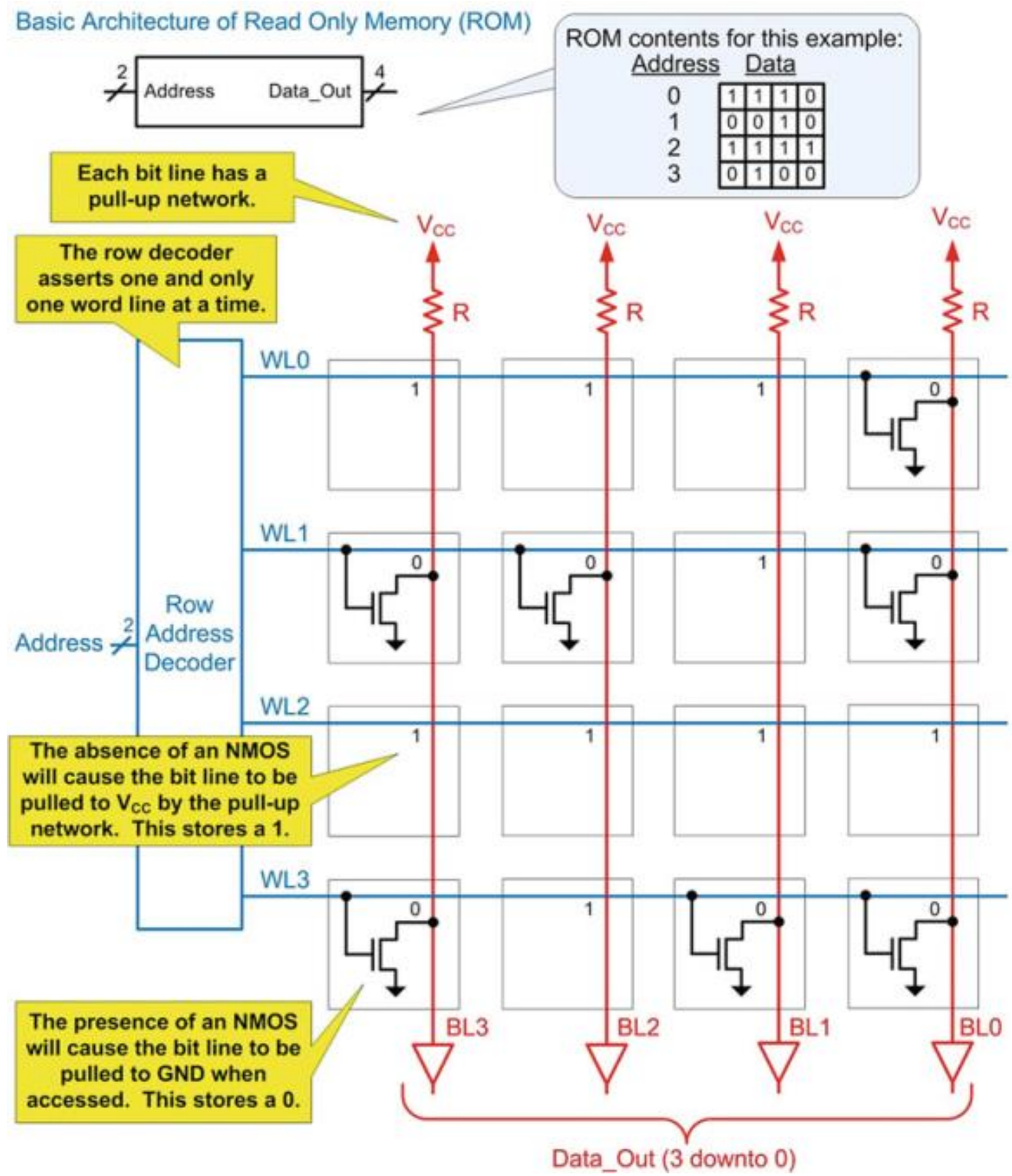
Table 7.3
ROM Truth Table (Partial)

Inputs					Outputs							
I_4	I_3	I_2	I_1	I_0	A_7	A_6	A_5	A_4	A_3	A_2	A_1	A_0
0	0	0	0	0	1	0	1	1	0	1	1	0
0	0	0	0	1	0	0	0	1	1	1	0	1
0	0	0	1	0	1	1	0	0	0	1	0	1
0	0	0	1	1	1	0	1	1	0	0	1	0
		⋮						⋮				
1	1	1	0	0	0	0	0	0	1	0	0	1
1	1	1	0	1	1	1	1	0	0	0	1	0
1	1	1	1	0	0	1	0	0	1	0	1	0
1	1	1	1	1	0	0	1	1	0	0	1	1



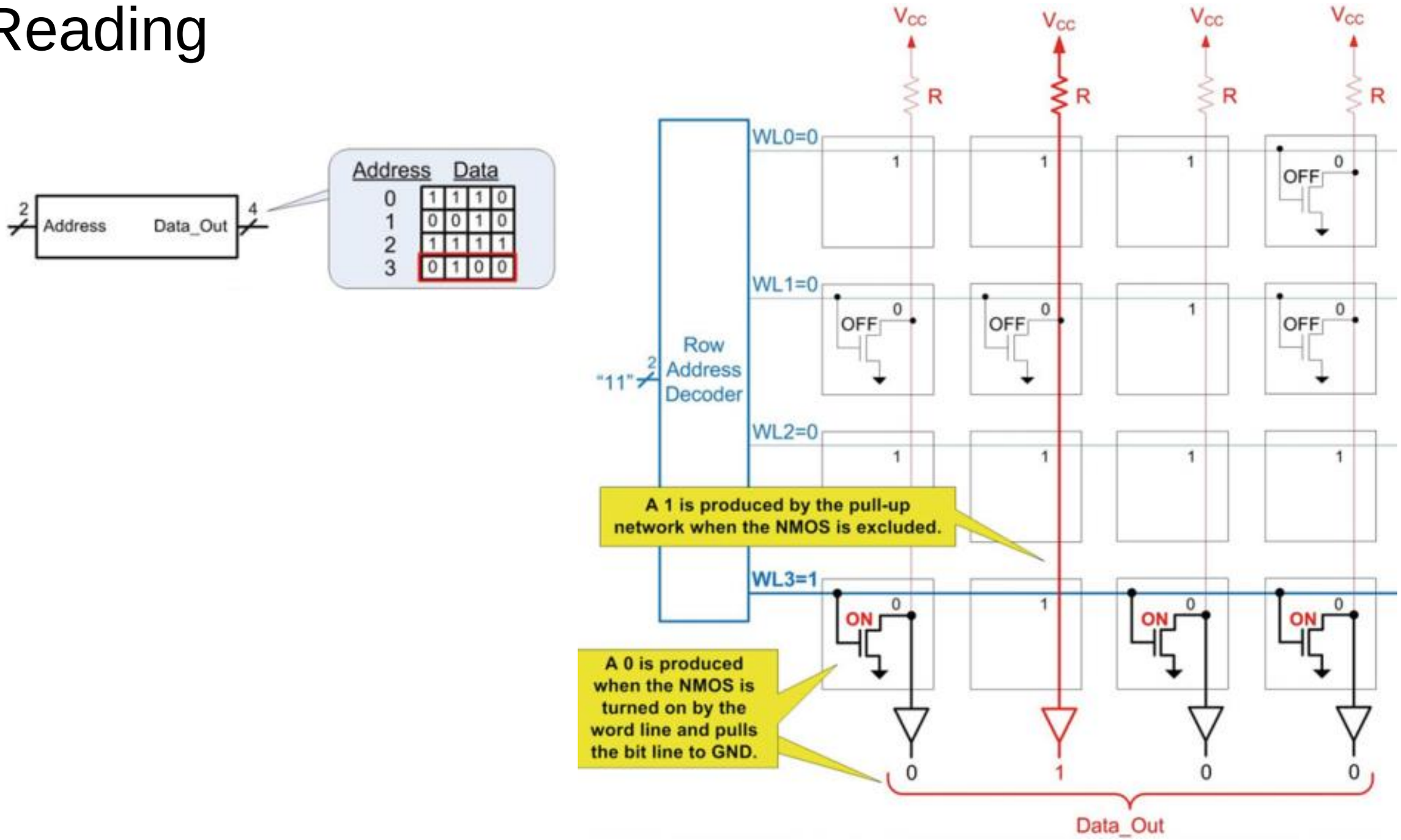
Read Only Memory (ROM)

Reading Structure

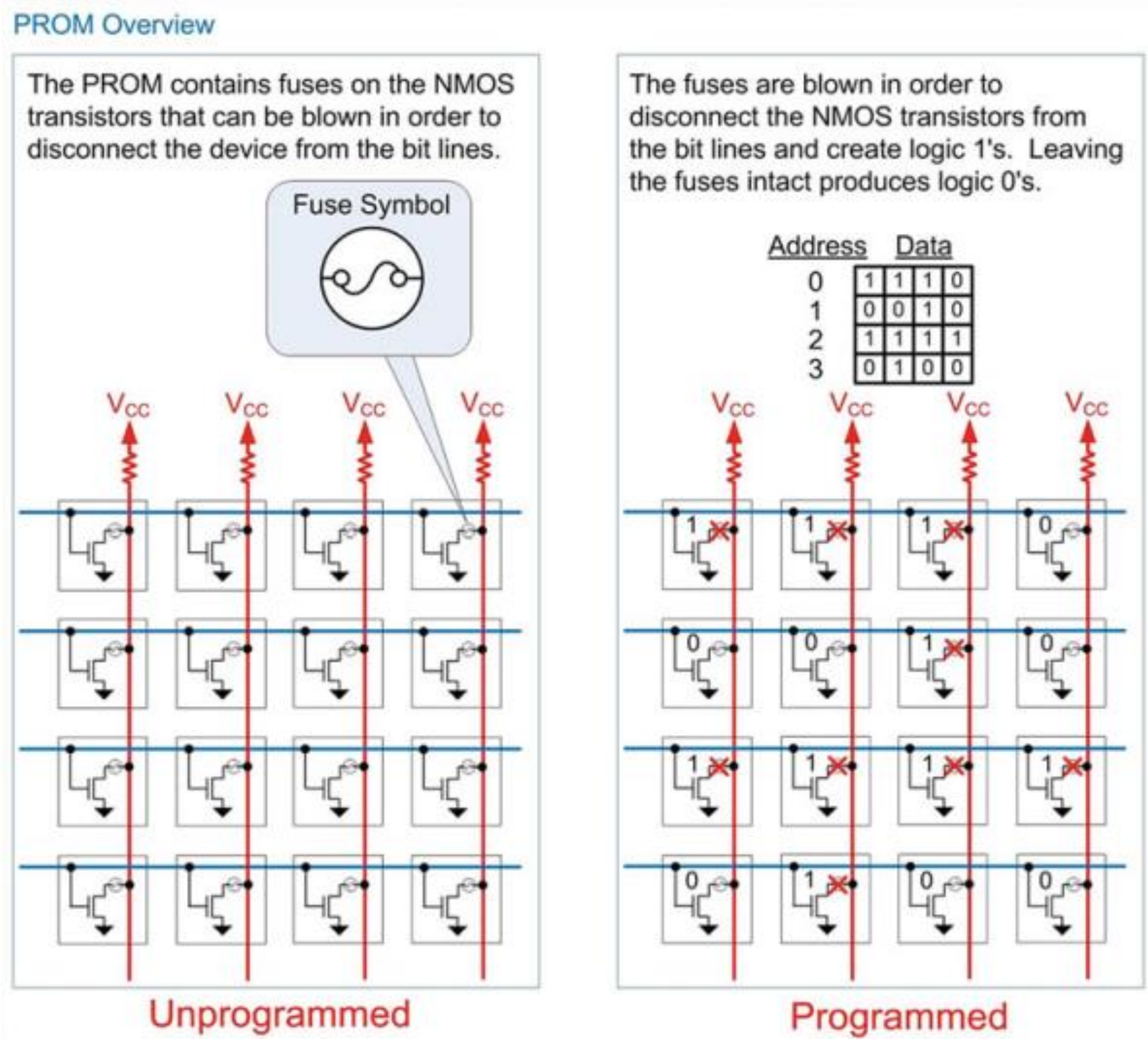


Read Only Memory (ROM)

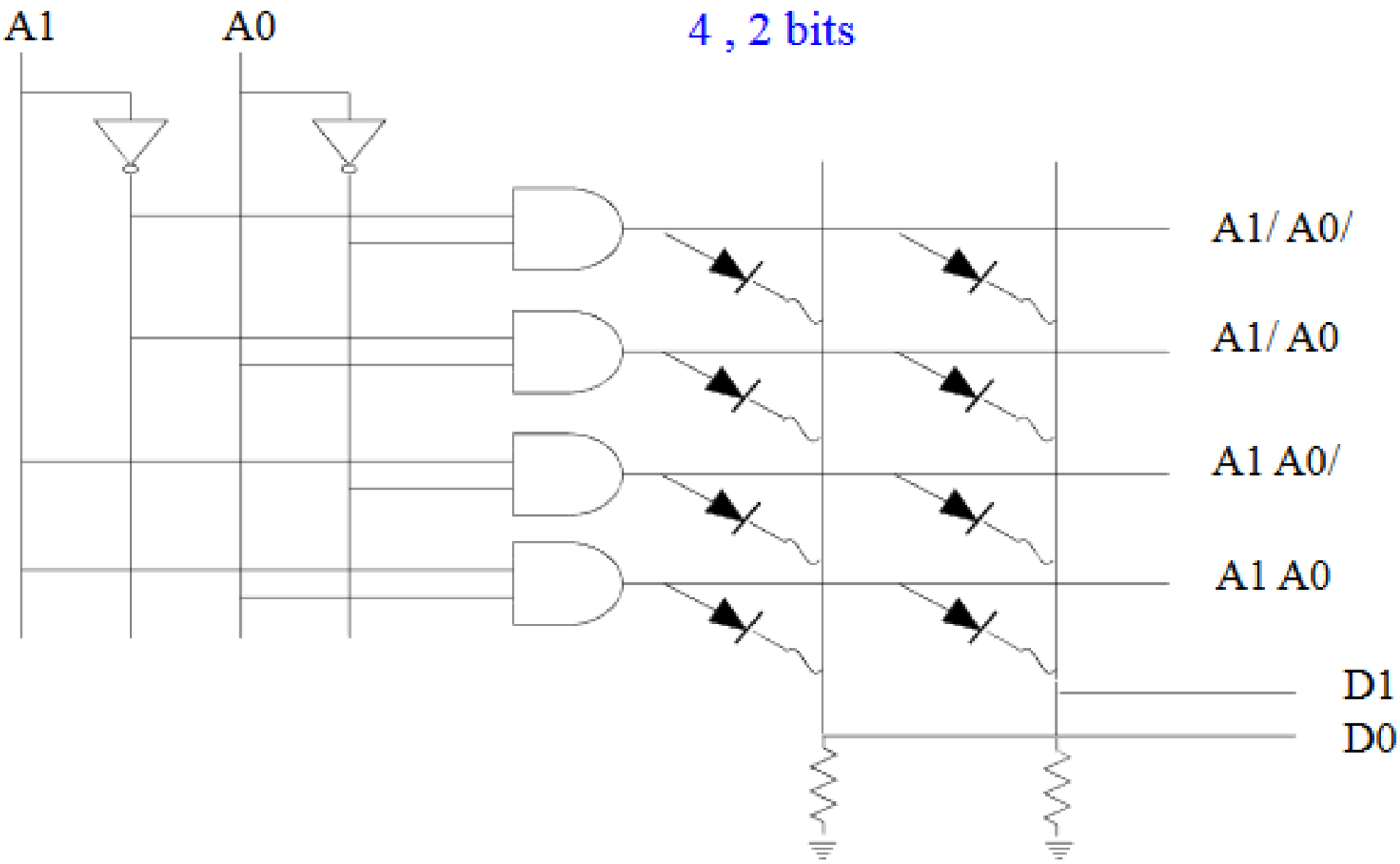
Reading



PROM


PROM Overview
The PROM contains fuses on the NMOS transistors that can be blown in order to disconnect the device from the bit lines.
Fuse Symbol
Vcc
Unprogrammed
The fuses are blown in order to disconnect the NMOS transistors from the bit lines and create logic 1's. Leaving the fuses intact produces logic 0's.
Address Data
0 1 1 1 0
1 0 0 1 0
2 1 1 1 1
3 0 1 0 0
Vcc
Programmed

PROM (Cont'd)



Fixed AND, Programmable OR

Combinational Circuit Implementation

- Similar to design procedure of circuits with decoders and OR gates as we have seen before
  - We have decoders and OR gates inside of the **ROM.**

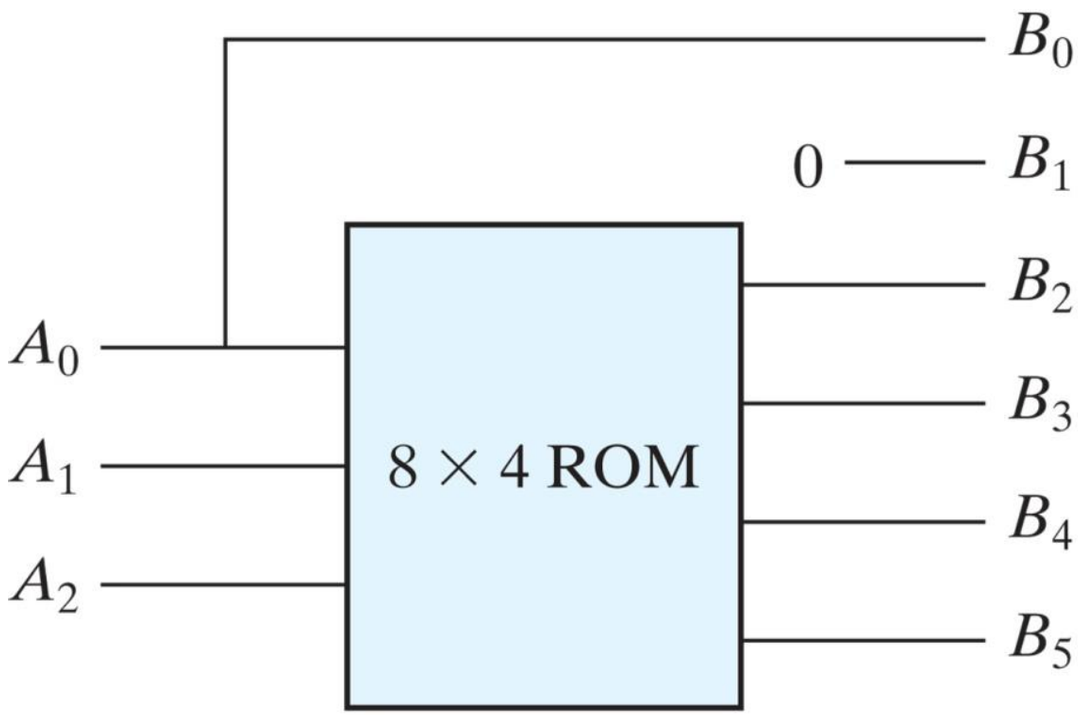
Design example

Design a combinational circuit using a ROM. The circuit accepts a 3-bit number and generates an output binary number equal to the square of the input number.

Table 7.4
Truth Table for Circuit of Example 7.1

Inputs			Outputs						
A_2	A_1	A_0	B_5	B_4	B_3	B_2	B_1	B_0	Decimal
0	0	0	0	0	0	0	0	0	0
0	0	1	0	0	0	0	0	1	1
0	1	0	0	0	0	1	0	0	4
0	1	1	0	0	1	0	0	1	9
1	0	0	0	1	0	0	0	0	16
1	0	1	0	1	1	0	0	1	25
1	1	0	1	0	0	1	0	0	36
1	1	1	1	1	0	0	0	1	49

- B1=0
- B0=A0



(a) Block diagram

A_2	A_1	A_0	B_5	B_4	B_3	B_2
0	0	0	0	0	0	0
0	0	1	0	0	0	0
0	1	0	0	0	0	1
0	1	1	0	0	1	0
1	0	0	0	1	0	0
1	0	1	0	1	1	0
1	1	0	1	0	0	1
1	1	1	1	1	0	0

(b) ROM truth table

Types of ROMs

- ROM-mask programming
  - Fill out the truth table, manufacturer makes the mask to produce 0's and 1's.
- PROM (programmable ROM)
  - Can be programmed in a lab by blowing the fuses (all fuses initially intact)
- EPROM (Erasable PROM)
  - Program it then erase under ultraviolet light.
- EEPROM or E^2PROM(Electrically erasable PROM)
  - Can be programmed and erased electrically.

Combinational PLDs

(a) Programmable read-only memory (PROM)

(b) Programmable array logic (PAL)

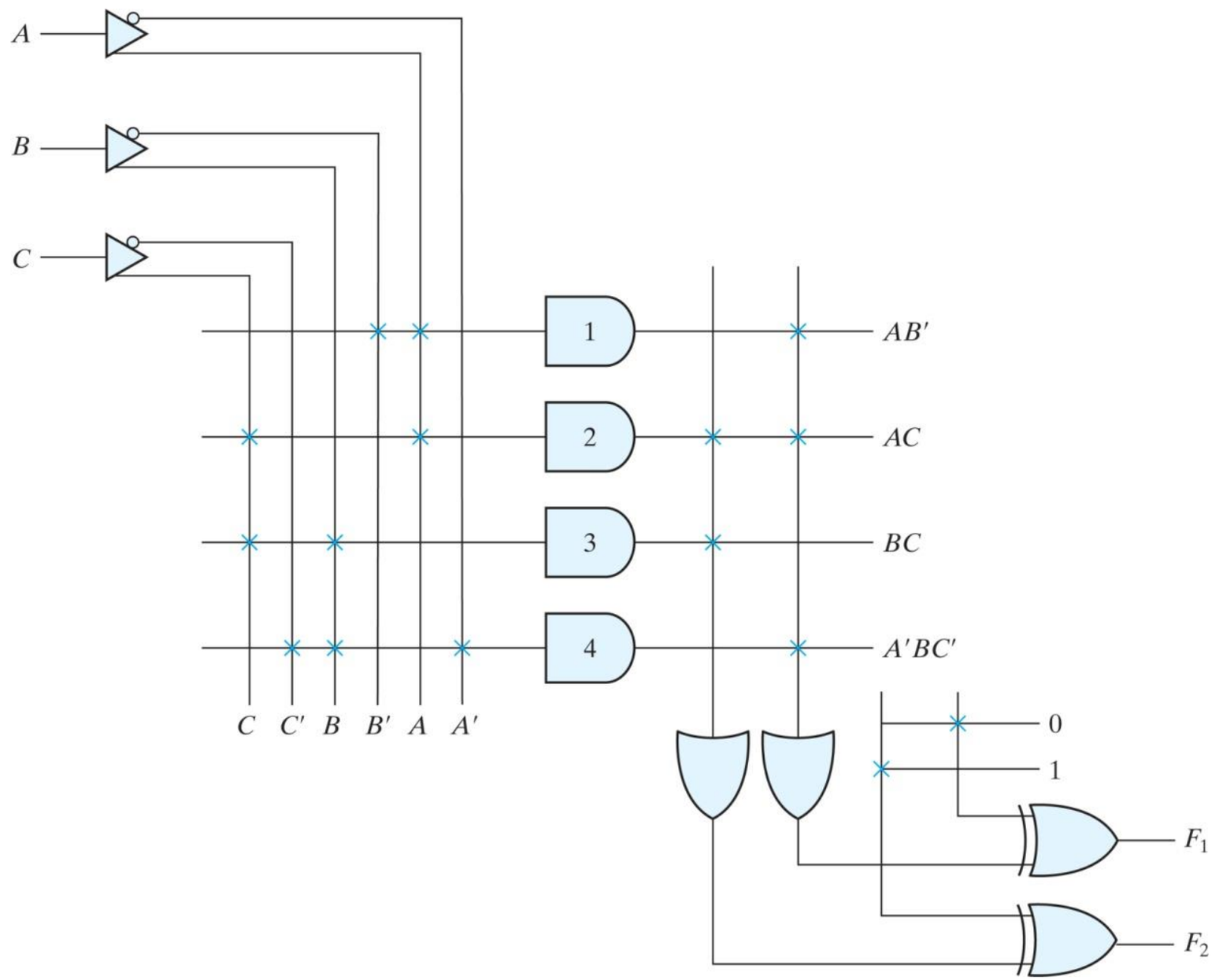
(c) Programmable logic array (PLA)

Programmable Logic Array

- Similar to PROMS except that decoder is replaced with AND gates.
- AND gates are connected to OR gates to produce sum-of-product terms.

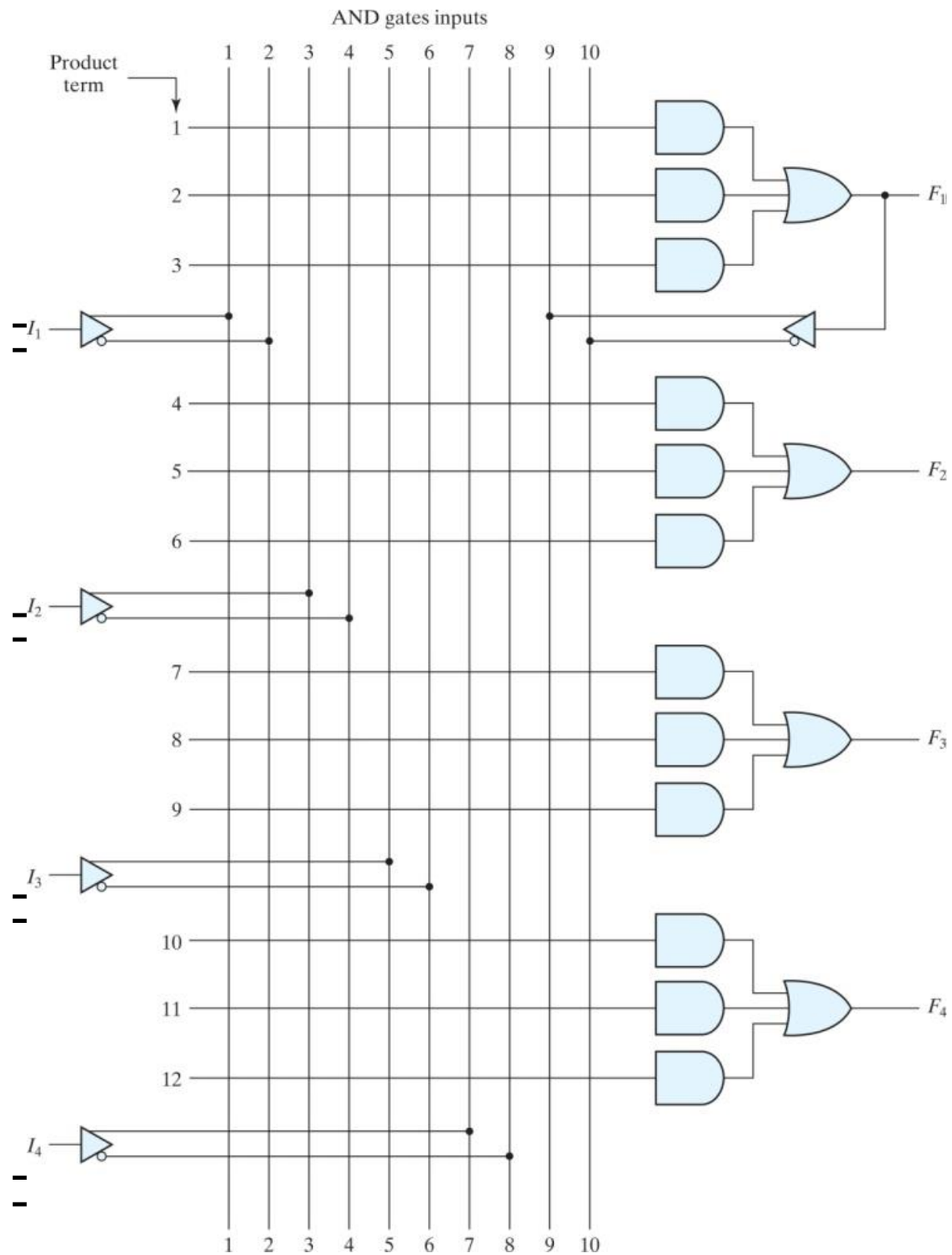
An example PLA circuit

- F1=AB'+AC+A'BC'
- F2=(AC+BC)'



Programmable Array Logic (PAL)

- Fixed OR array and programmable AND array.
- Figure shows 4 input 4 output PAL



Design example

As an example of using a PAL in the design of a combinational circuit, consider the following Boolean fu11ctions, given in sum of minterms:

$$w(A, B, C, D) = \Sigma(2, 12, 13)$$
$$x(A, B, C, D) = \Sigma(7, 8, 9, 10, 11, 12, 13, 14, 15)$$
$$y(A, B, C, D) = \Sigma(0, 2, 3, 4, 5, 6, 7, 8, 10, 11, 15)$$
$$z(A, B, C, D) = \Sigma(1, 2, 8, 12, 13)$$

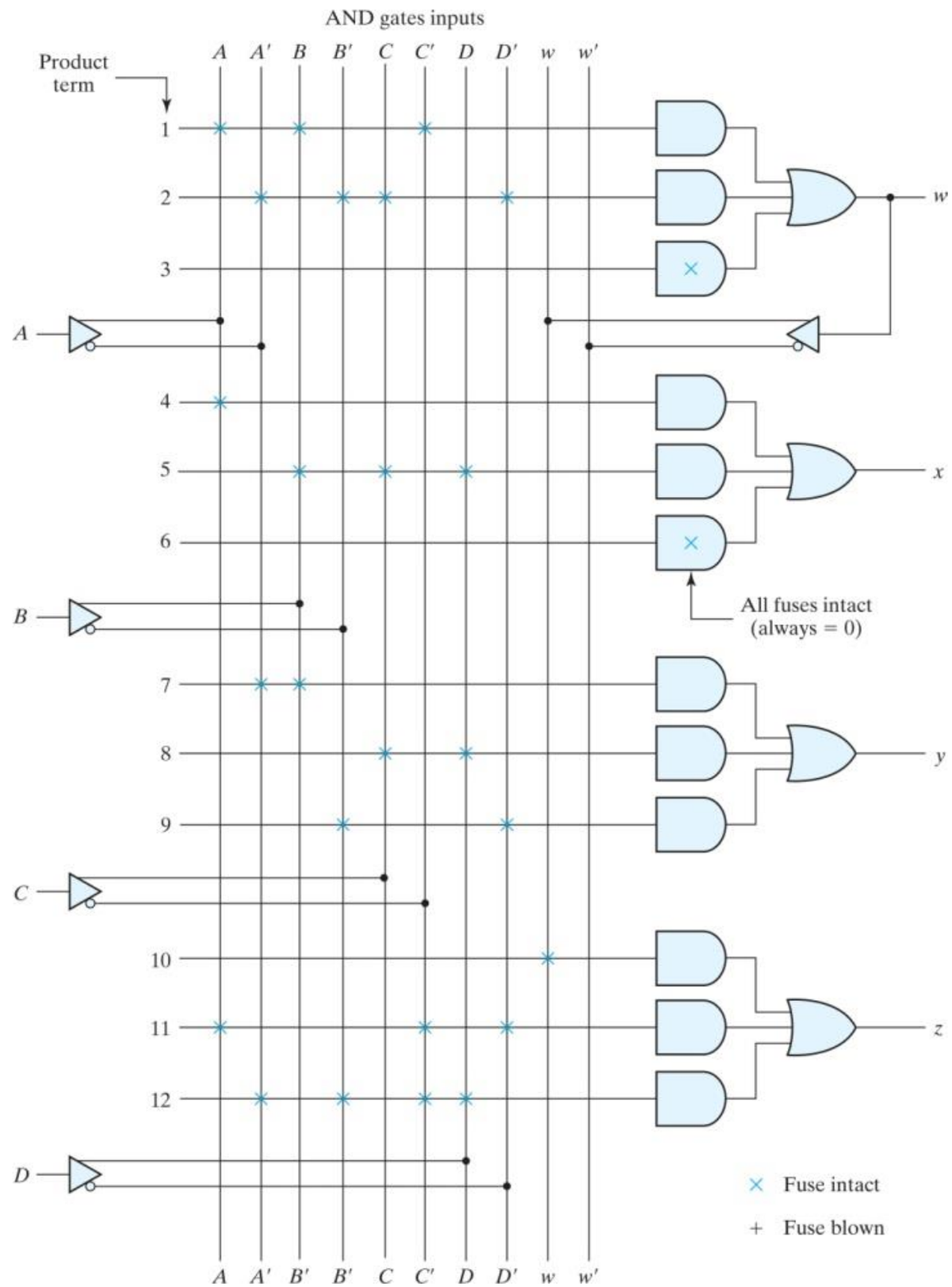
Simplifying the four functions to a minimum number of terms results in the following Boolean functions:

$$w = ABC' + A'B'CD'$$
$$x = A + BCD$$
$$y = A'B + CD + B'D'$$
$$z = ABC' + A'B'CD' + AC'D' + A'B'C'D$$
$$= w + AC'D' + A'B'C'D$$

- PALs may need simplifications as *z* includes *w* in this example.

Table 7.6
PAL Programming Table

Product Term	AND Inputs					Outputs
	A	*B*	*C*	*D*	*w*	
1	1	1	0	—	—	$w = ABC' + A'B'CD'$
2	0	0	1	0	—	
3	—	—	—	—	—	
4	1	—	—	—	—	$x = A + BCD$
5	—	1	1	1	—	
6	—	—	—	—	—	
7	0	1	—	—	—	$x = A'B + CD' + B'D'$
8	—	—	1	1	—	
9	—	0	—	0	—	
10	—	—	—	—	1	$z = w + AC'D' + A'B'C'D$
11	1	—	0	0	—	
12	0	0	0	1	—	

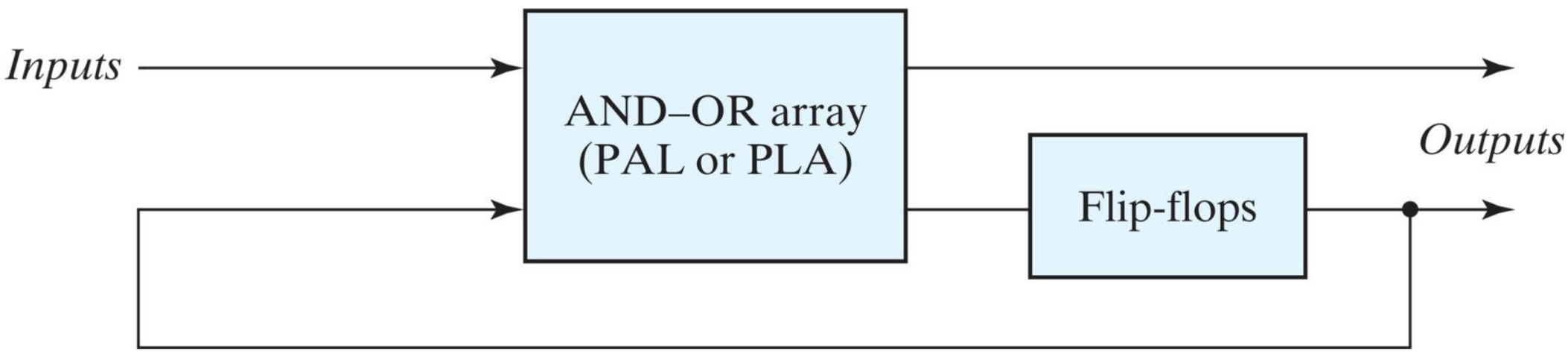

AND gates inputs
A A′ B B′ C C′ D D′ w w′
Product term
1
2
3
4
5
6
7
8
9
10
11
12
A
B
C
D
w
x
y
z
All fuses intact (always = 0)
× Fuse intact
+ Fuse blown

Sequential Programmable Devices

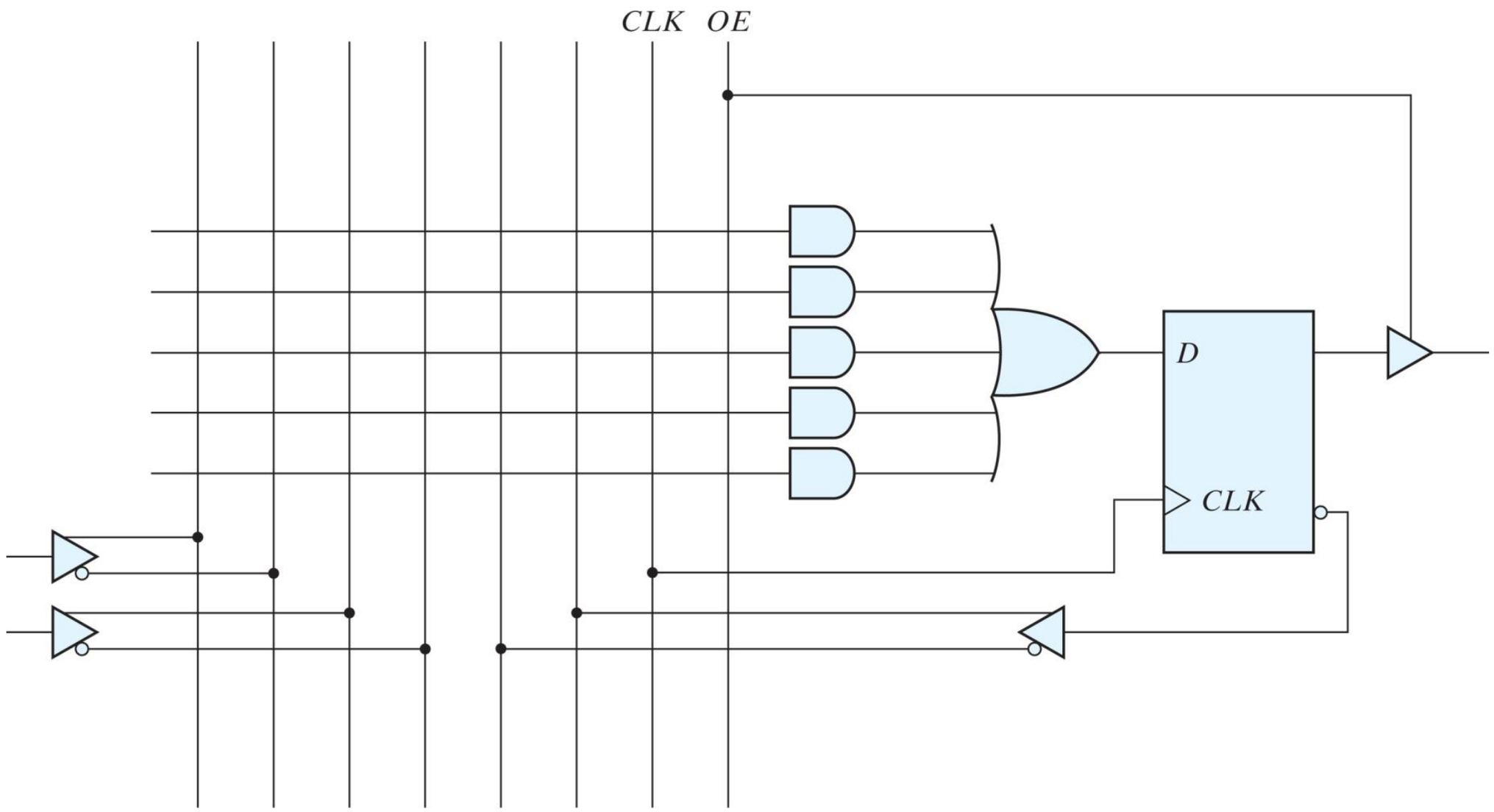
- Needs flip-flops
- Types

1. Sequential (or simple) Programmable Logic Device (SPLD)
2. Complex Programmable Logic Device (CPLD)
3. Field Programmable Gate Array (FPGA)

SPLD

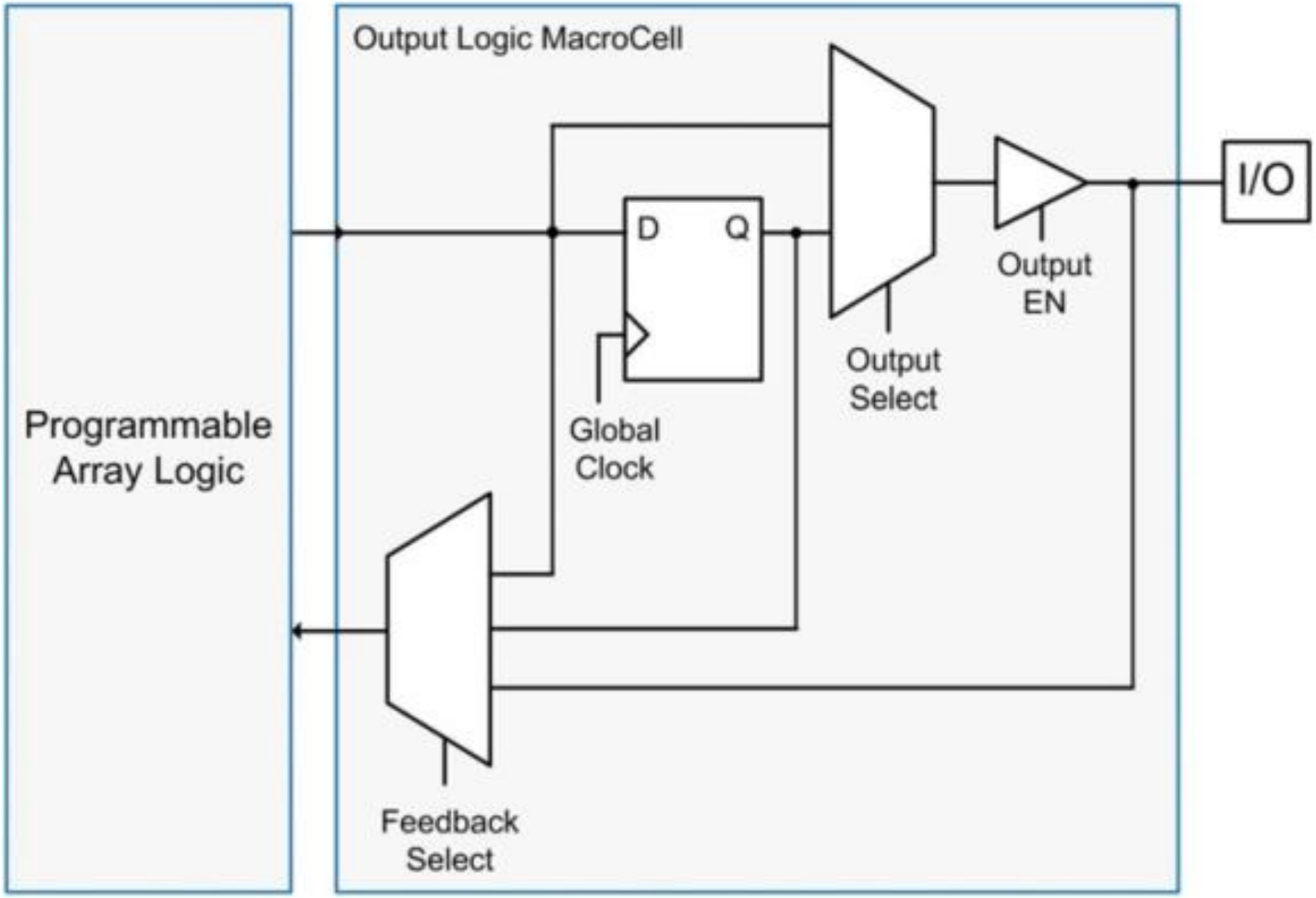


Basic Macrocell Logic



Generic Array Logic (GAL)

Adding an output logic macrocell to a PAL allows the system to implement feedback, create sequential logic, or use the I/O pin as either an input or output.



Evolution

PROM: Programmable Read Only Memory

- Fixed AND, Programmable OR

PLA: Programmable Logic Array

- Programmable AND, Programmable OR

PAL: Programmable Array Logic

- Programmable AND, Fixed OR

Simple Programmable Logic Device

Complex Programmable Logic Device

Field Programmable Gate Array

References

Disclaimer: I don’t claim the ownership of all the slides. Some of the slides are shamelessly copied/adopted from various publicly available sources on the internet. ☺

- Brock J. LaMeres “Introduction to logic circuits & Logic Design with Verilog”, Chapter 10 & 11